

Esquema de mi DB en SQLite

```
sqlite>
sqlite> .tables
202030851_clientes      202030851_detalle_pedido  202030851_pedidos      202030851_productos
sqlite>
sqlite>
sqlite>
sqlite> .schema "202030851_clientes"
CREATE TABLE "202030851_clientes" (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nombre TEXT NOT NULL,
  correo TEXT UNIQUE
);
sqlite> █
```

Tablas Creadas en SQLite

```
202030851_clientes      202030851_detalle_pedido  202030851_pedidos      202030851_productos
sqlite>
sqlite>
sqlite>
sqlite> .schema "202030851_clientes"
CREATE TABLE "202030851_clientes" (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nombre TEXT NOT NULL,
  correo TEXT UNIQUE
);
sqlite> .schema "202030851_detalle_pedido"
CREATE TABLE "202030851_detalle_pedido" (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  pedido_id INTEGER,
  producto_id INTEGER,
  cantidad INTEGER,
  FOREIGN KEY (pedido_id) REFERENCES "202030851_pedidos"(id),
  FOREIGN KEY (producto_id) REFERENCES "202030851_productos"(id)
);
sqlite> .schema "202030851_pedidos"
CREATE TABLE "202030851_pedidos" (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  cliente_id INTEGER,
  fecha TEXT,
  FOREIGN KEY (cliente_id) REFERENCES "202030851_clientes"(id)
);
sqlite> .schema "202030851_productos"
CREATE TABLE "202030851_productos" (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nombre TEXT NOT NULL,
  precio REAL NOT NULL
);
sqlite> █
```

Resultado del Select con los datos recién ingresados sin hacerle Update

```
(3) 2023-07-29 7)
sqlite> INSERT INTO "202030851_detalle_pedido" (pedido_id, producto_id, cantidad) VALUES
(1, 1, 1),
(1, 2, 2),
(2, 3, 1),
(3, 4, 1),
(4, 5, 3);
sqlite> SELECT * FROM "202030851_clientes";
```

id	nombre	correo
1	Juan Perez	juan@email.com
2	Maria Lopez	maria@email.com
3	Carlos Gomez	carlos@email.com
4	Ana Torres	ana@email.com
5	Luis Ramirez	luis@email.com

```
sqlite> SELECT * FROM "202030851_productos";
```

id	nombre	precio
1	Laptop	800.5
2	Mouse	15.0
3	Teclado	25.0
4	Monitor	150.75
5	USB	10.0

```
sqlite> SELECT * FROM "202030851_detalle_pedido";
```

id	pedido_id	producto_id	cantidad
1	1	1	1
2	1	2	2
3	2	3	1
4	3	4	1
5	4	5	3

```
sqlite> █
```

Resultado del Select con los datos actualizados.

```
-- 3. Cambiar cantidad en detalle
```

```
UPDATE "202030851_detalle_pedido"
```

```
SET cantidad = 5
```

```
WHERE id = 2;
```

```
sqlite> SELECT * FROM "202030851_clientes";
```

id	nombre	correo
1	Juan Actualizado	juan@email.com
2	Maria Lopez	maria@email.com
3	Carlos Gomez	carlos@email.com
4	Ana Torres	ana@email.com
5	Luis Ramirez	luis@email.com

```
sqlite> SELECT * FROM "202030851_productos";
```

id	nombre	precio
1	Laptop	900.0
2	Mouse	15.0
3	Teclado	25.0
4	Monitor	150.75
5	USB	10.0

```
sqlite> SELECT * FROM "202030851_detalle_pedido";
```

id	pedido_id	producto_id	cantidad
1	1	1	1
2	1	2	5
3	2	3	1
4	3	4	1
5	4	5	3

```
sqlite> █
```



```

db > database > \r tareaDB1.sql

-- Tabla de clientes
CREATE TABLE "202030851_clientes" (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL,
    correo TEXT UNIQUE
);

-- Tabla de productos
CREATE TABLE "202030851_productos" (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL,
    precio REAL NOT NULL
);

-- Tabla de pedidos
CREATE TABLE "202030851_pedidos" (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    cliente_id INTEGER,
    fecha TEXT,
    FOREIGN KEY (cliente_id) REFERENCES "202030851_clientes"(id)
);

-- Tabla de detalle de pedidos
CREATE TABLE "202030851_detalle_pedido" (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    pedido_id INTEGER,
    producto_id INTEGER,
    cantidad INTEGER,
    FOREIGN KEY (pedido_id) REFERENCES "202030851_pedidos"(id),
    FOREIGN KEY (producto_id) REFERENCES "202030851_productos"(id)
);

INSERT INTO "202030851_clientes" (nombre, correo) VALUES
('Juan Perez', 'juan@email.com'),
('Maria Lopez', 'maria@email.com'),
('Carlos Gomez', 'carlos@email.com'),
('Ana Torres', 'ana@email.com'),
('Luis Ramirez', 'luis@email.com');

INSERT INTO "202030851_productos" (nombre, precio) VALUES
('Laptop', 800.50),
('Mouse', 15.00),
('Teclado', 25.00),
('Monitor', 150.75),
('USB', 10.00);

INSERT INTO "202030851_pedidos" (cliente_id, fecha) VALUES
(1, '2026-04-27'),
(2, '2026-04-26'),
(3, '2026-04-25'),
(4, '2026-04-24'),
(5, '2026-04-23');

INSERT INTO "202030851_detalle_pedido" (pedido_id, producto_id, cantidad) VALUES
(1, 1, 1),
(1, 2, 2),
(2, 3, 1),
(3, 4, 1),
(4, 5, 3);

-- 1. Cambiar nombre de cliente

```

```

1  (1, 3, 5);
2  -- 1. Cambiar nombre de cliente
3  UPDATE "202030851_clientes"
4  SET nombre = 'Juan Actualizado'
5  WHERE id = 1;
6
7  -- 2. Cambiar precio de producto
8  UPDATE "202030851_productos"
9  SET precio = 900.00
10 WHERE id = 1;
11
12 -- 3. Cambiar cantidad en detalle
13 UPDATE "202030851_detalle_pedido"
14 SET cantidad = 5
15 WHERE id = 2;
16
17
18
19 SELECT * FROM "202030851_clientes";
20 SELECT * FROM "202030851_detalle_pedido";
21 SELECT * FROM "202030851_detalle_pedido";

```

Preguntas:

¿Qué es una base de datos embebida y en qué se diferencia de una base de datos cliente-servidor?

Una **base de datos embebida** es una base de datos que está **integrada directamente dentro de una aplicación**. No necesitas instalar un servidor aparte porque todo funciona dentro del mismo programa.

En cambio, una **base de datos cliente-servidor** funciona con **dos partes separadas**:

- Un **servidor** (donde está la base de datos)
- Uno o varios **clientes** (aplicaciones que se conectan a ese servidor)

¿Qué archivo o carpeta genera tu motor al crear una base de datos y qué contiene?

Cuando se trabaja con **SQLite**, hay algo importante que entender, todo vive en archivos dentro de tu sistema.

¿Qué archivo genera SQLite?

Cuando creas una base de datos en SQLite, normalmente se genera:

Un solo archivo, por ejemplo: mi_DB.db

¿Qué contiene ese archivo?

Este archivo guarda **todo lo necesario** de la base de datos:

- **Tablas** (estructura de datos)
- **Registros** (los datos en sí)

- **Relaciones**
- **Índices**
- **Metadatos** (información sobre cómo está organizada la BD)

¿Qué tipos de datos soporta tu motor? Menciona al menos 5 y da un ejemplo de uso para cada uno

En SQLite los tipos de datos funcionan de forma un poco distinta a otros motores: usa un sistema de **tipado dinámico** basado en “afinidades de tipo”. Aun así, hay tipos comunes que se usan en la práctica.

Aquí tienes al menos 5 tipos con ejemplos claros:

1. INTEGER

Almacena números enteros.

Ejemplo:

```
CREATE TABLE usuarios (  
  id INTEGER PRIMARY KEY,  
  edad INTEGER  
);
```

2. REAL

Números decimales (punto flotante).

Ejemplo:

```
CREATE TABLE productos (  
  precio REAL  
);
```

3. TEXT

Cadenas de texto.

Ejemplo:

```
CREATE TABLE clientes (  
  nombre TEXT,  
  correo TEXT  
);
```

4. BLOB

Datos binarios (imágenes, archivos, etc.).

Ejemplo:

```
CREATE TABLE archivos (  
    datos BLOB  
);
```

5. NULL

Representa ausencia de valor.

Ejemplo:

```
INSERT INTO usuarios (id, edad) VALUES (1, NULL);
```

¿Qué ocurre si ejecutas un UPDATE sin cláusula WHERE? Demuéstralo con un ejemplo

Si ejecutas un UPDATE **sin cláusula** WHERE, el motor SQLite aplicará el cambio **a todas las filas de la tabla**. No hay filtro, así que cada registro es actualizado.

Ejemplo: todos los datos fueron actualizados.

```
sqlite> UPDATE "202030851_clientes"  
SET nombre = 'Juan Actualizado';  
sqlite> SELECT * FROM "202030851_clientes";
```

id	nombre	correo
1	Juan Actualizado	juan@email.com
2	Juan Actualizado	maria@email.com
3	Juan Actualizado	carlos@email.com
4	Juan Actualizado	ana@email.com
5	Juan Actualizado	luis@email.com

```
sqlite> █
```

¿Qué es una restricción NOT NULL y cuándo deberías usarla?

Una restricción **NOT NULL** en SQLite es una regla que impide que una columna tenga valores NULL. Es decir, **obliga a que siempre exista un valor** en ese campo.

¿En qué escenario elegirías tu motor por encima de los otros vistos en laboratorio/clase?

En aplicaciones móviles locales o proyectos pequeños que necesitan guardar información donde nos sea necesario instalar un motor de db y que la misma app lo esté manipulando datos en tiempo real.